

Autor: Cezary Bogusławski 198036

SPRAWOZDANIE Z PROJEKTU: SORTOWANIE METODĄ DUŻYCH BUFORÓW

Temat: Implementacja i analiza efektywności algorytmu scalania z użyciem wielkich buforów.

1. Opis zastosowanej metody

W projekcie zaimplementowano metodę sortowania zewnętrznego przez scalanie z użyciem wielkich buforów. Rozwiązanie symuluje pracę na taśmach magnetycznych przy użyciu standardowych plików dyskowych.

Struktura i organizacja:

- Pliki: System wykorzystuje 3 pliki fizyczne:
 1. data.txt – plik wejściowy z danymi nieposortowanymi.
 2. runs.txt – plik roboczy przechowujący serie (tzw. runs).
 3. out.txt – plik wyjściowy/tymczasowy wykorzystywany podczas scalania.
- Warstwa logiczna: Zaimplementowano klasę FileManager, która symuluje stronicowanie pamięci. Warstwa ta udostępnia operacje odczytu i zapisu pojedynczych rekordów, tłumacząc je na operacje blokowe.
- Parametry pamięci:
 - Rozmiar strony (blok): $b = 10$ rekordów.
 - Liczba buforów w pamięci operacyjnej: $N = 10$.

Algorytm działania:

1. Faza dystrybucji (Tworzenie serii): Dane są wczytywane z pliku wejściowego do bufora w pamięci operacyjnej. Następuje ich posortowanie wewnętrzne, a następnie zapisanie posortowanej serii do pliku roboczego.
2. Faza scalania: Następuje k-ścieżkowe scalanie (gdzie $k = N = 10$). Program pobiera strony z k serii, wybiera element najmniejszy i zapisuje go do bufora wyjściowego. Po wyczerpaniu serii następuje przejście do kolejnej fazy, aż do uzyskania jednej, w pełni posortowanej serii.
3. Specyfikacja formatu pliku testowego

Plik testowy jest plikiem tekstowym, w którym każdy wiersz reprezentuje pojedynczy rekord.

- Typ rekordu: Wektor 4-wymiarowy.
- Reprezentacja: Cztery liczby zmiennoprzecinkowe oddzielone spacjami.
- Kryterium sortowania: Długość (magnituda) wektora liczona od początku układu współrzędnych (norma euklidesowa).

Przykład formatu danych:

-10.8 0.444 1.2 10.83

5.12 -2.0 0.0 1.1

3. Opis sposobu prezentacji wyników

Program komunikuje się z użytkownikiem poprzez konsolę tekstową. Po uruchomieniu oferuje tryb testowy (automatyczny eksperyment) lub tryb pojedynczego sortowania.

Podczas działania i po zakończeniu wyświetlane są:

- Zawartość pliku przed i po posortowaniu.
- Liczba faz sortowania.
- Statystyki operacji wejścia/wyjścia:
 - Liczba odczytów stron dyskowych.
 - Liczba zapisów stron dyskowych.
 - Łączny koszt (suma odczytów i zapisów).
- Porównanie kosztu rzeczywistego z wyliczonym kosztem teoretycznym.

4. Opis eksperymentu

Cel i metodyka:

Celem eksperymentu było zbadanie złożoności operacyjnej zaimplementowanego algorytmu w zależności od liczby sortowanych rekordów (N) oraz weryfikacja zgodności wyników z modelem teoretycznym.

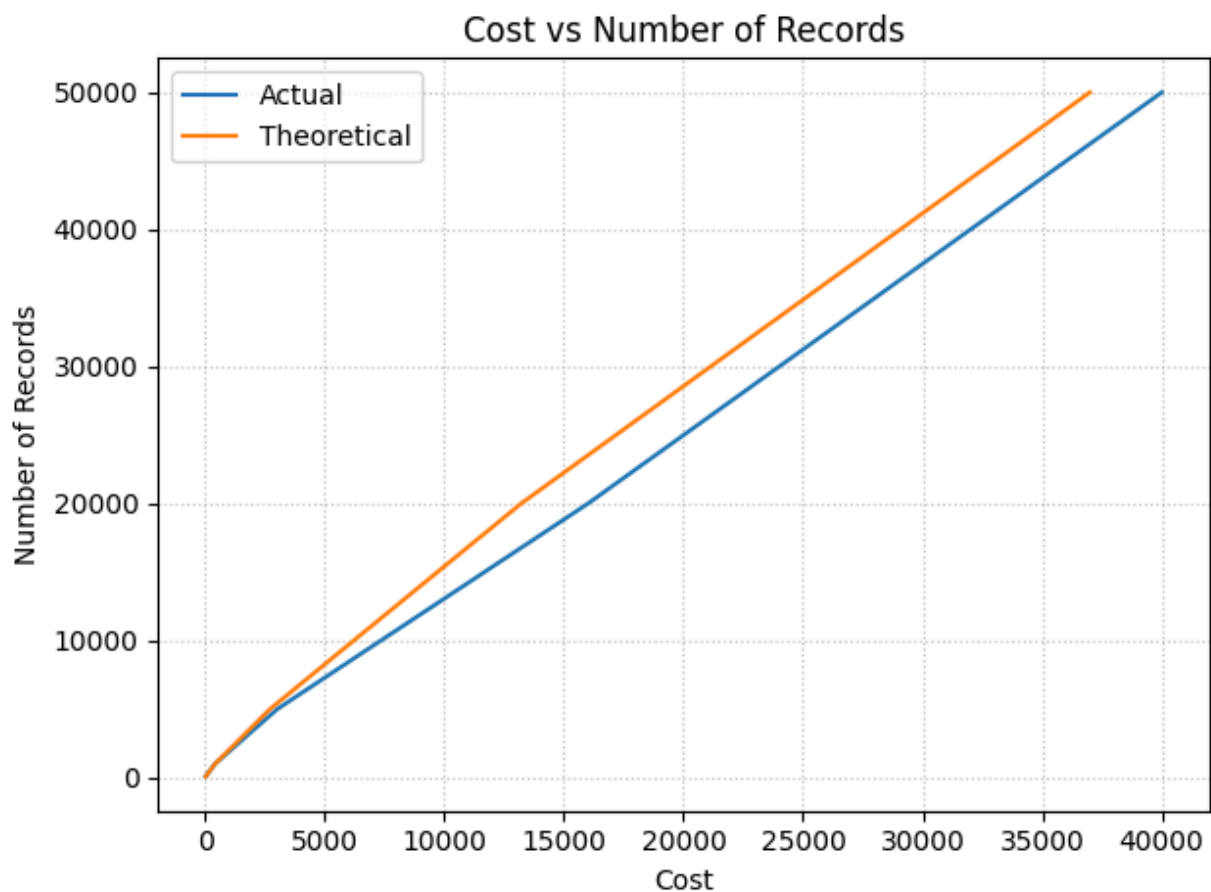
Badanie przeprowadzono dla 5 różniących się wielkości zbiorów danych: 100, 1000, 5000, 20000 oraz 50000 rekordów. Dla każdego przypadku zliczano fizyczne operacje zapisu i odczytu stron.

Wyniki liczbowe eksperymentu:

Liczba Rekordów (N)	Odczyty stron	Zapisy stron	Łączny Koszt (R+W)	Koszt Teoretyczny	Liczba Faz
100	10	10	20	20	0
1000	200	200	400	400	1
5000	1500	1500	3000	2699	2
20000	8000	8000	16000	13204	3
50000	20000	20000	40000	36990	3

Wykres:

Wykres łącznej liczby operacji dyskowych w funkcji liczby rekordów (N) przedstawia linię niebieską (koszt rzeczywisty) oraz linię pomarańczową (koszt teoretyczny).



Komentarz do wyników i porównanie z teorią:

1. Charakterystyka wzrostu: Zarówno wyniki teoretyczne, jak i praktyczne potwierdzają, że koszt sortowania zewnętrznego rośnie w tempie zbliżonym do liniowo-logarytmicznego. Jest to zgodne z teoretyczną złożonością metody, gdzie liczba operacji zależy od liczby rekordów oraz logarytmu o podstawie liczby buforów.
2. Analiza rozbieżności:
 - Dla mniejszych zbiorów danych (100, 1000 rekordów) wyniki teoretyczne pokrywają się z praktycznymi.
 - Dla większych zbiorów (od 5000 rekordów) zauważalna jest tendencja, w której koszt rzeczywisty jest wyższy od teoretycznego. Różnica ta wynika ze specyfiki sortowania zewnętrznego, które działa w dyskretnych fazach. Wzór teoretyczny zastosowany w programie zakłada "idealny", ciągły przyrost kosztu. W rzeczywistości, jeśli dane nie mieszczą się idealnie w pełnych seriach dla danej fazy, konieczne jest rozpoczęcie kolejnej, pełnej fazy przetwarzania całego pliku, co skokowo zwiększa liczbę operacji odczytu i zapisu.
3. Wnioski: Algorytm działa poprawnie. Różnice między modelem teoretycznym a praktyką są przewidywalne i wynikają z zaokrągleń liczby faz scalania do liczb całkowitych w górę (funkcja sufit), podczas gdy model teoretyczny w tym przypadku jest funkcją wygładzoną.